

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report
import seaborn as sns
```

```
(x_train_full, y_train_full), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train_full = x_train_full.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

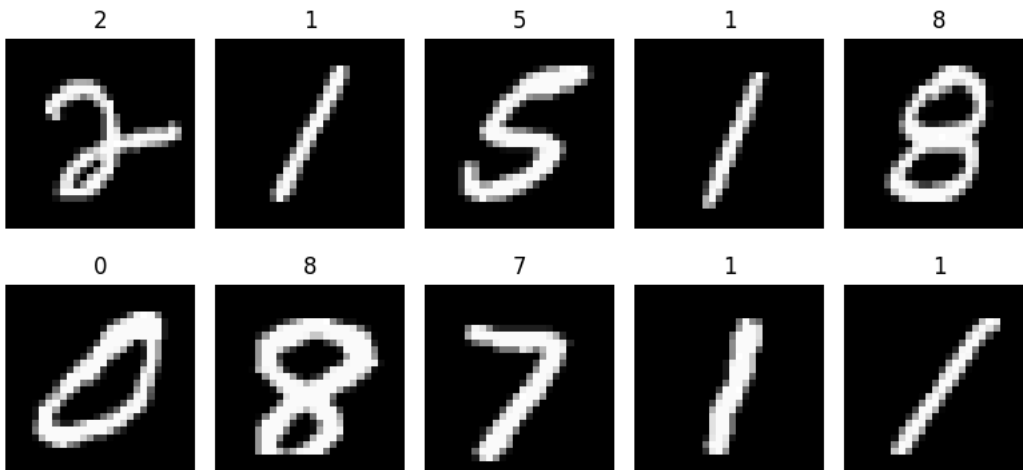
x_train_full = x_train_full.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

x_train, x_val, y_train, y_val = train_test_split(
    x_train_full, y_train_full, test_size=0.1, random_state=42, stratify=y_train_full
)

print(x_train.shape, x_val.shape, x_test.shape)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 — 1s 0us/step
(54000, 28, 28, 1) (6000, 28, 28, 1) (10000, 28, 28, 1)

```
plt.figure(figsize=(8, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i].reshape(28, 28), cmap="gray")
    plt.title(str(y_train[i]))
    plt.axis("off")
plt.tight_layout()
plt.show()
```



```
model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(10, activation="softmax")
])

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
conv2d_1 (Conv2D)	(None, 24, 24, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 12, 12, 32)	0
dropout (Dropout)	(None, 12, 12, 32)	0
conv2d_2 (Conv2D)	(None, 10, 10, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
dropout_1 (Dropout)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout_2 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

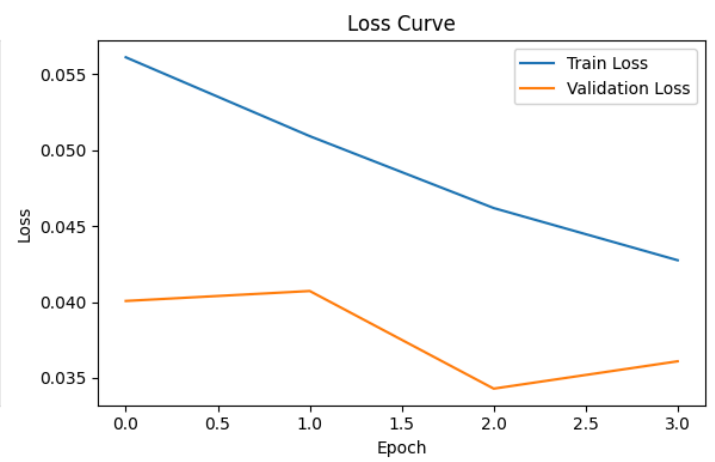
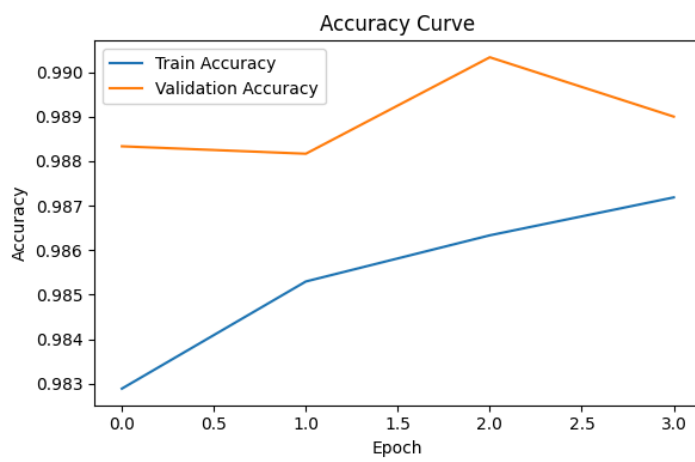
Total params: 234,282 (915.16 KB)
Trainable params: 234,282 (915.16 KB)
Non-trainable params: 0 (0.00 B)

```
model.compile(  
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

```
history = model.fit(  
    x_train, y_train,  
    validation_data=(x_val, y_val),  
    epochs=4,  
    batch_size=128,  
    verbose=1  
)
```

Epoch 1/4
422/422 ————— 104s 247ms/step - accuracy: 0.9829 - loss: 0.0561 - val_accuracy: 0.9883 - val_loss: 0.0401
Epoch 2/4
422/422 ————— 136s 233ms/step - accuracy: 0.9853 - loss: 0.0509 - val_accuracy: 0.9882 - val_loss: 0.0407
Epoch 3/4
422/422 ————— 98s 233ms/step - accuracy: 0.9863 - loss: 0.0462 - val_accuracy: 0.9903 - val_loss: 0.0343
Epoch 4/4
422/422 ————— 99s 234ms/step - accuracy: 0.9872 - loss: 0.0428 - val_accuracy: 0.9890 - val_loss: 0.0361

```
plt.figure(figsize=(12, 4))  
  
plt.subplot(1, 2, 1)  
plt.plot(history.history["accuracy"], label="Train Accuracy")  
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")  
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.title("Accuracy Curve")  
plt.legend()  
  
plt.subplot(1, 2, 2)  
plt.plot(history.history["loss"], label="Train Loss")  
plt.plot(history.history["val_loss"], label="Validation Loss")  
plt.xlabel("Epoch")  
plt.ylabel("Loss")  
plt.title("Loss Curve")  
plt.legend()  
  
plt.tight_layout()  
plt.show()
```



```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print("Test Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_acc, 4))
```

Test Loss: 0.0242
Test Accuracy: 0.9924

```
y_pred_prob = model.predict(x_test, verbose=0)
y_pred = np.argmax(y_pred_prob, axis=1)

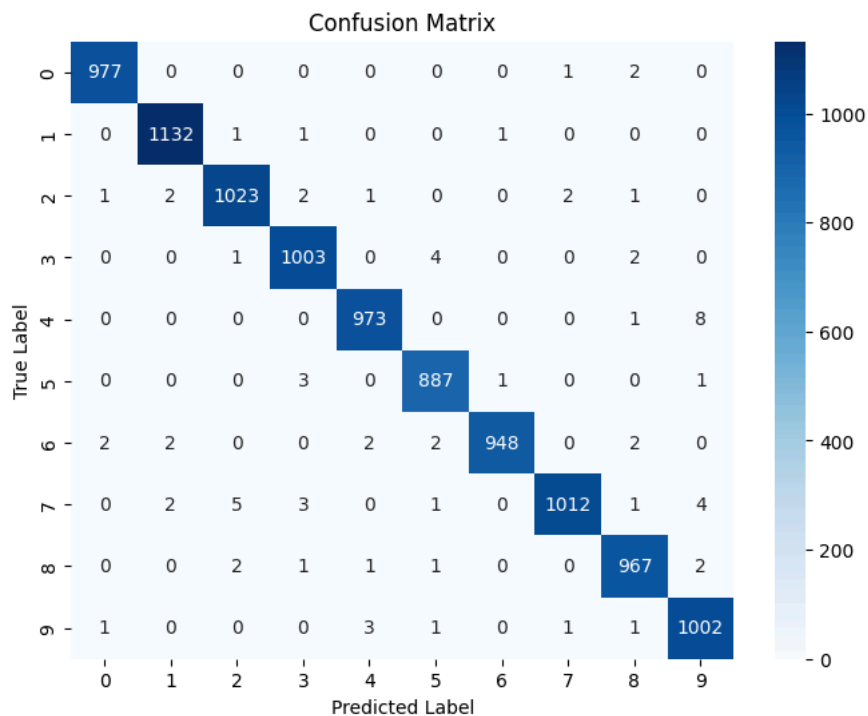
print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(y_test, y_pred, average="macro"), 4))
print("Recall   :", round(recall_score(y_test, y_pred, average="macro"), 4))
print("F1-Score :", round(f1_score(y_test, y_pred, average="macro"), 4))
print()
print(classification_report(y_test, y_pred, digits=4))
```

Accuracy : 0.9924
Precision: 0.9924
Recall : 0.9924
F1-Score : 0.9924

	precision	recall	f1-score	support
0	0.9959	0.9969	0.9964	980
1	0.9947	0.9974	0.9960	1135
2	0.9913	0.9913	0.9913	1032
3	0.9901	0.9931	0.9916	1010
4	0.9929	0.9908	0.9918	982
5	0.9900	0.9944	0.9922	892
6	0.9979	0.9896	0.9937	958
7	0.9961	0.9844	0.9902	1028
8	0.9898	0.9928	0.9913	974
9	0.9853	0.9931	0.9891	1009
accuracy			0.9924	10000
macro avg	0.9924	0.9924	0.9924	10000
weighted avg	0.9924	0.9924	0.9924	10000

```
cm = confusion_matrix(y_test, y_pred)

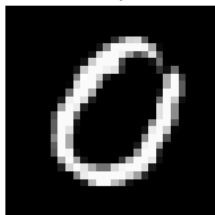
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=range(10), yticklabels=range(10))
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()
```



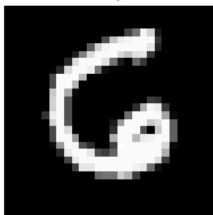
```
indices = np.random.choice(len(x_test), 12, replace=False)

plt.figure(figsize=(12, 6))
for i, idx in enumerate(indices):
    plt.subplot(3, 4, i + 1)
    plt.imshow(x_test[idx].reshape(28, 28), cmap="gray")
    plt.title("Actual: " + str(y_test[idx]) + " | Pred: " + str(y_pred[idx]))
    plt.axis("off")
plt.tight_layout()
plt.show()
```

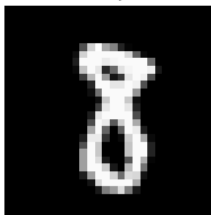
Actual: 0 | Pred: 0



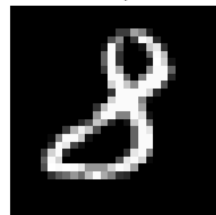
Actual: 6 | Pred: 6



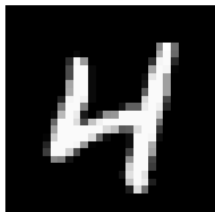
Actual: 8 | Pred: 8



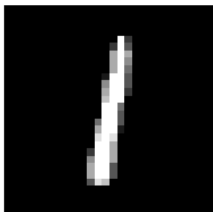
Actual: 8 | Pred: 8



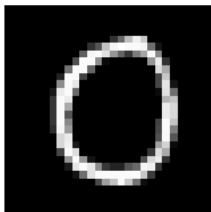
Actual: 4 | Pred: 4



Actual: 1 | Pred: 1



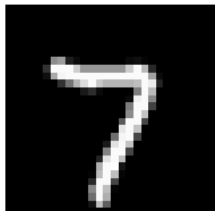
Actual: 0 | Pred: 0



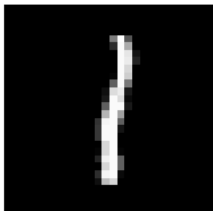
Actual: 5 | Pred: 5



Actual: 7 | Pred: 7



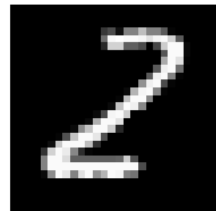
Actual: 1 | Pred: 1



Actual: 4 | Pred: 4



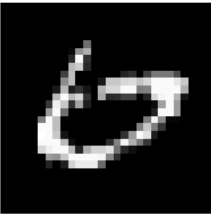
Actual: 2 | Pred: 2



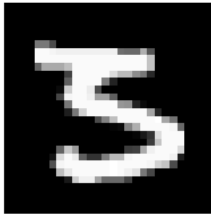
```
wrong = np.where(y_pred != y_test)[0]
print("Total Misclassified:", len(wrong))
```

```
plt.figure(figsize=(12, 6))
for i, idx in enumerate(wrong[:12]):
    plt.subplot(3, 4, i + 1)
    plt.imshow(x_test[idx].reshape(28, 28), cmap="gray")
    plt.title("Actual: " + str(y_test[idx]) + " | Pred: " + str(y_pred[idx]))
    plt.axis("off")
plt.tight_layout()
plt.show()
```

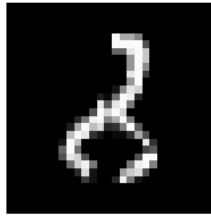
Actual: 6 | Pred: 0



Actual: 3 | Pred: 5



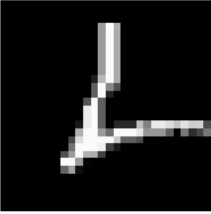
Actual: 8 | Pred: 2



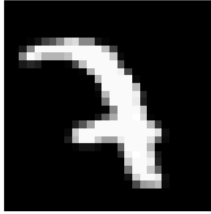
Actual: 6 | Pred: 4



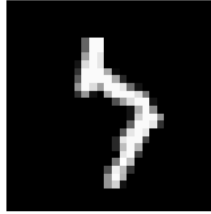
Actual: 2 | Pred: 4



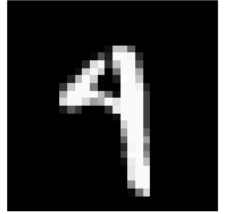
Actual: 7 | Pred: 3



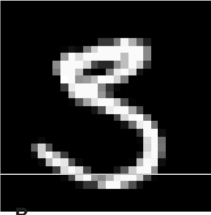
Actual: 7 | Pred: 5



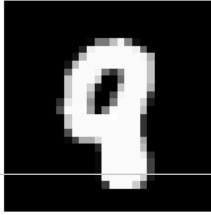
Actual: 4 | Pred: 9



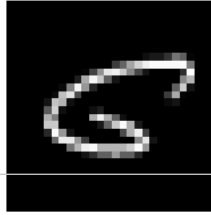
Actual: 3 | Pred: 5



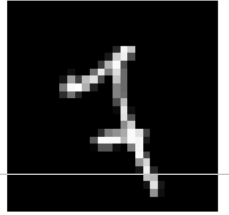
Actual: 8 | Pred: 9



Actual: 6 | Pred: 5



Actual: 7 | Pred: 3

**PART - B**

1. What is image classification? How does it differ from object detection and image segmentation? Image classification assigns one label to the whole image. Object detection finds objects with bounding boxes and labels. Segmentation labels every pixel individually.
2. Explain the architecture and working principle of a Convolutional Neural Network (CNN). A CNN passes an image through conv → activation → pooling layers to extract features, then through fully connected layers for classification. It uses shared filters, making it efficient at learning spatial patterns automatically.
3. What is the role of Convolutional Layers, Pooling Layers, and Fully Connected Layers in a CNN? Conv layers extract features using filters. Pooling layers reduce size and computation. Fully connected layers use these features to make the final prediction.
4. Why is image normalization performed before training a deep learning model? Normalization scales pixels to 0–1, keeping training stable and fast. It prevents exploding/vanishing gradients and helps the model converge better.
5. Explain the purpose of activation functions such as ReLU and Softmax in CNNs. ReLU adds non-linearity, letting the network learn complex patterns. Softmax converts final outputs into class probabilities summing to 1.
6. What is a Confusion Matrix? How is it used to evaluate classification performance? A confusion matrix compares actual vs predicted labels in a table. It shows correct predictions and exactly which classes are being confused with each other.